

Архітектурний аналіз та стратегія еволюції платформи aegisroute-ui-starter до системи інтелектуального рефакторингу коду на базі DRAKON

Глибокий інженерний та архітектурний аналіз поточної кодової бази проєкту aegisroute-ui-starter, а також зіставлення її з фундаментальними специфікаціями екосистеми візуального програмування DRAKON, виявляє необхідність парадигмального зсуву в підходах до генерації діаграм. Перехід від базового інструментарію візуалізації до повноцінної платформи інтелектуального рефакторингу, яка здатна на двосторонню синхронізацію між вихідним кодом і діаграмою за допомогою великих мовних моделей (LLM), вимагає впровадження архітектури гібридної топологічно-семантичної екстракції (Hybrid Topological Semantic Extraction, HTSE).¹ Цей звіт надає вичерпний структурний, топологічний та архітектурний аналіз наявної системи, а також містить деталізований план її технологічної еволюції у вигляді поетапної дорожньої карти.

1. Аналіз кодової бази Lovable-проєкту

Поточна кодова база побудована на сучасному стеку технологій, що забезпечує високу продуктивність рендерингу та гнучкість маршрутизації, проте виявляє суттєві архітектурні прогалини в контексті інтеграції детермінованих алгоритмів аналізу коду та роботи зі штучним інтелектом.

1.1. Frontend Architecture

Фронтенд-частина реалізована як Single Page Application (SPA) на базі React 19, використовуючи екосистему TanStack, яка включає Router, Query та Start, у поєднанні зі складальником Vite.¹ Використання @tanstack/react-router забезпечує типобезпечну маршрутизацію через файли src/router.tsx та src/app/AppRouter.tsx, що є стандартом для сучасних складних вебзастосунків.¹ Глобальний стан управління діаграмами раціонально ізолювано в Zustand-сховищі src/store/useDiagramStore.ts, яке інкапсулює поточну діаграму, метрики генерації, такі як індикатор єдиного шампура (sis), глибину вкладеності (rdc), загальну кількість вузлів, а також флаги збереження та наявності незбережених змін.¹

Інтерфейс користувача та шар візуальних компонентів побудовано на базі Radix UI примітивів (shadcn/ui) з використанням Tailwind CSS для стилізації.¹ Наявні складні компоненти, такі як DrakonEditor.tsx, DrakonViewer.tsx та FormatInspector.tsx, забезпечують повноцінне середовище для взаємодії користувача з графом, дозволяючи редагувати

контент, формувати вузли та переглядати діаграми у режимі читання.¹ Особливої уваги заслуговує управління подіями у `DrakonEditor.tsx`, де використовується механізм `useStateRef` для відстеження станів контекстного меню та режиму вставлення (`paste mode`), що запобігає перехопленню подій миші фреймворком `React` і дозволяє `vanilla-JS` віджету коректно обробляти кліки на полотні.¹

1.2. Backend / Worker Architecture

Бекенд реалізовано у вигляді безсерверної функції (`Serverless`) за допомогою `Cloudflare Workers`, де файл `cloudflare-worker/worker-mcp-drakon.js` слугує єдиною точкою входу для API-запитів.¹ Аналіз показує, що поточна реалізація орієнтована виключно на виконання базових `CRUD`-операцій з документами. Воркер безпосередньо взаємодіє з об'єктним сховищем, сумісним зі стандартом `S3 (MinIO)`, використовуючи внутрішні методи для завантаження, отримання та видалення файлів, зберігаючи діаграми у форматі `JSON` з прив'язкою до ідентифікатора папки та ідентифікатора самої діаграми за ключем формату `drakon/${folderSlug}/${diagramId}.json`.¹ Хоча така архітектура забезпечує високу масштабованість для зберігання статичних даних, вона повністю позбавлена обчислювальних потужностей для виконання парсингу абстрактних синтаксичних дерев (`AST`) або складного семантичного аналізу коду, що є фундаментальною вимогою для систем інтелектуального рефакторингу.¹

1.3. Auth Flow

Система авторизації реалізує підхід на основі `JWT (JSON Web Tokens)`, що є типовим для розподілених застосунків. На клієнтському рівні файл `src/lib/auth.ts` керує збереженням токена у `localStorage`, а стан авторизації інтегровано в `React`-дерево через контекст провайдер `AuthContext.tsx`.¹ Маршрути захищаються хуком `useRequireAuth`, який перехоплює неавторизовані сесії та перенаправляє користувачів на сторінку логіну.¹ На серверному рівні `Cloudflare Worker` генерує `JWT` з терміном дії 24 години на базі секретного ключа з оточення.¹

Проте зберігання `JWT` безпосередньо у `localStorage` створює критичну вразливість до атак типу `Cross-Site Scripting (XSS)`. Для платформи рефакторингу коду, де користувачі можуть завантажувати, генерувати або переглядати потенційно небезпечні фрагменти вихідного коду, експлуатація `XSS` дозволить зловмиснику миттєво викрасти токен авторизації. Це вимагає негайного переходу на використання механізмів `HttpOnly Secure cookies` для управління сесіями.

1.4. Diagram Storage Flow

Потік збереження даних використовує гібридну модель, поєднуючи локальний кеш та віддалену синхронізацію. Утиліта `src/lib/diagram-storage.ts` дублює кожен мутувану діаграму в `localStorage` (за ключем `drakon.diagrams`), що забезпечує базові можливості для роботи без інтернет-з'єднання та гарантує швидке початкове завантаження інтерфейсу.¹ При ініціації збереження через виклик методу `saveDiagram` у глобальному стані

useDiagramStore.ts, фронтенд відправляє мутований JSON на бекенд через метод `api.commit`, після чого оновлює локальний кеш та скидає флаг наявності незбережених змін.¹ Однак цей процес є повністю синхронним і не враховує можливості паралельного редагування, що створює ризик втрати даних у випадках, коли над однією діаграмою одночасно працюватимуть людина та автономний AI-агент.

1.5. MCP Tool Layer

Протокол Model Context Protocol (MCP) частково імплементовано на рівні Cloudflare Worker.¹ Сервер експонує чотири базові інструменти стандарту JSON-RPC: `drakon.list_diagrams`, `drakon.get_diagram`, `drakon.save_diagram` та `drakon.delete_diagram`.¹ Цей набір інструментів покриває лише операції вводу/виводу файлової системи, що є еквівалентним базовому MCP Filesystem серверу. Архітектурна прогалина полягає у повній відсутності семантичних інструментів для витягування абстрактних синтаксичних дерев, валідації топології або внесення точкових ітеративних мутацій у структуру коду.¹ Без цих інструментів великі мовні моделі не зможуть виконувати структурний рефакторинг, оскільки вони змушені будуть перезаписувати весь файл діаграми цілком, що неминуче призведе до руйнування топології на великих алгоритмах.¹

1.6. Drakon Editor Integration

Інтеграція візуальної мови DRAKON реалізована шляхом глибокого обгортання сторонньої `vanilla-JS` бібліотеки `drakonwidget.js` у React-компоненти. Бібліотека завантажується динамічно через утиліту `loadDrakonWidget` у файлі `src/lib/drakon/adapter.ts` та ініціалізується в ізолюваному DOM-контейнері за допомогою `useRef`.¹ Взаємодія зі станом відбувається через об'єкт `editSender`, який передається до віджета та перехоплює всі мутації графа, маркуючи локальний стан `Zustand` як модифікований.¹ Тематизація ефективно вирішена у файлі `themeAdapter.ts`, який конвертує системні теми додатка у специфічний формат `DrakonConfigTheme`, зрозумілий віджету.¹ Такий підхід до інтеграції є архітектурно правильним, оскільки він зберігає високу продуктивність детермінованого макетування `vanilla-JS` рушія, відокремлюючи його від циклу рендерингу React.¹

1.7. Current Generation Flow

Поточний процес перетворення тексту або коду в діаграму реалізовано за антипатерном "чорного ящика". У файлі `DiagramsPage.tsx` користувач вводить код або текстовий опис, який відправляється на ендпоінт `api.generate`.¹ Бекенд повертає готовий JSON-об'єкт діаграми, масив авто виправлень та метрики структури, включаючи індикатор дотримання правила шампура та глибину алгоритму.¹ Такий підхід свідчить про те, що вся логіка генерації, визначення зв'язків та макетування наразі покладена виключно на LLM. Згідно з інженерними специфікаціями DRAKON, великі мовні моделі не мають внутрішнього двовимірного рендеру та математичної здатності до безпомилкового топологічного розміщення, що неминуче призводитиме до "просторових галюцинацій", накладання вузлів

та візуального хаосу при обробці складних алгоритмічних структур.¹

1.8. Weak Points and Missing Modules

Аналіз виявляє критичні слабкі місця, які унеможливають використання системи у production середовищі для складного рефакторингу коду. По-перше, системі критично бракує детермінованого AST-парсера (такого як ts-morph або tree-sitter-graph), що унеможливорює точний статичний аналіз вхідного коду перед його передачею до LLM.¹ По-друге, незважаючи на наявність спостережливих метрик, у системі відсутній незалежний мікросервіс Graph Validator, який би програмно блокував перетини ліній, усував вузли-сироти або виправляв заборонені лівобічні вектори.¹ По-третє, архітектура мутацій є повністю синхронною і позбавленою FIFO-черги (First-In-First-Out). У сучасних AI-платформах ітеративні мутації вимагають суворої серіалізації для уникнення стану гонитви (race condition), коли і людина, і агент намагаються змінити вузол одночасно.¹

2. Карта модулів і залежностей

Наведена нижче таблиця ілюструє структурну мапу ключових модулів системи, їхнє безпосереднє призначення та міжкомпонентні залежності. Вона також відображає поточний статус готовності кожного модуля до інтеграції в цільову HTSE-архітектуру для побудови AI-to-DRAKON конвеєра.

Шар (Layer)	Модуль / Файл	Призначення у системі	Залежності	Статус для цільової архітектури
View (React UI)	DrakonEditor.tsx	Обгортка віджета, обробка подій миші/клавіатур, управління діалогами редагування контенту та стилів.	drakonwidget.js, useDiagramStore.ts, FormatInspector.tsx	Адаптувати: Додати обробку ітеративних AI-ін'єкцій та візуалізацію диференціалів (diffs).
View (React UI)	DiagramsPage.tsx	Керування списком схем, виклик процесу генерації діаграм з коду	api.ts, diagram-store.ts	Адаптувати: Повна переробка для підтримки асинхронного HTSE-конвеєра

		або тексту.		а замість синхронного <code>api.generate</code> .
State Management	<code>useDiagramStore.ts</code>	Управління локальним станом діаграми за допомогою Zustand, застосування структурних дельт.	<code>api.commit</code> , <code>zustand</code>	Адаптувати: Імплементувати і серіалізацію FIFO для узгодження паралельних змін від MCP та UI.
DRAKON Core	<code>drakonwidget.js</code>	Детермінований алгоритм геометричного макетування, рендеринг, Манхеттенська маршрутизація ліній.	Відсутні (Vanilla JS)	Зберегти: Ідеальний компонент, що гарантує відсутність перетинів та підтримує плоский JSON. ¹
Data Adapters	<code>drakongen.js / pseudocode.ts</code>	Зворотна трансляція JSON-структури діаграми у псевдокод або текстовий формат Markdown.	<code>drakonwidget.js</code> (схема IR)	Адаптувати: Розширити можливості до повноцінної генерації синтаксичних дерев (AST) цільових мов.
API & Storage	<code>api.ts</code> , <code>diagram-storage.ts</code>	Клієнтський шар для взаємодії з Cloudflare Worker та локальним <code>localStorage</code>	<code>fetch</code> API	Зберегти: Стабільний шар, що потребує лише розширення типів даних

		кешем.		для нових ендпоінтів.
Backend / Worker	worker-mcp-drakon.js	Обробка протоколу MCP, аутентифікація, взаємодія з об'єктним сховищем S3 (MinIO).	Cloudflare Workers API, S3 API	Адаптувати: Розширити функціонал додаванням нових MCP інструментів для доступу до AST та автофіксу.
Code Analysis	[Відсутній модуль]	Детермінована трансляція TypeScript/JS коду в Control Flow Graph (CFG).	ts-morph або tree-sitter	Побудувати: Критично необхідний новий мікросервіс для першого етапу HTSE конвеєра. ¹
Validation Layer	[Відсутній модуль]	Перевірка топологічних інваріантів (відсутність перетинів, єдиний термінал) та алгоритмічний автофікс.	Специфікація DRAKON IR	Побудувати: Детермінований валідатор для обробки згенерованого LLM словника перед рендерингом. ¹

Детальний аналіз графа залежностей демонструє високий ступінь зчеплення (coupling) між логікою генерації та клієнтським станом. Це призводить до ситуації, коли будь-яка зміна в алгоритмі вимагає перезапису всього стану в Zustand, що робить неможливою гранульовану роботу AI-агентів.

3. Оцінка готовності до Production: Reuse / Adapt / Build

Для успішного перетворення наявної системи на платформу інтелектуального рефакторингу, здатну витримувати навантаження production середовища, необхідно чітко

розмежувати існуючий код за стратегією утилізації. Деякі компоненти є зрілими і можуть бути використані негайно, тоді як інші вимагають глибокої адаптації або створення з нуля.

3.1. Що вже готове для production use (Reuse)

Абсолютно готовим до використання без жодних архітектурних змін є ядро візуалізації `drakonwidget.js`. Аналіз екосистеми підтверджує, що цей компонент бездоганно вирішує найскладнішу математичну задачу візуальної складової — Манхеттенську маршрутизацію ліній без перетинів та дотримання жорсткої вертикальної гравітації.¹ Його здатність оперувати плоским JSON-словником робить його ідеальним термінальним вузлом для AI-to-DRAKON конвеєра.¹ Також готовою до масштабування є клієнтська інфраструктура на базі React 19 та екосистеми TanStack, яка забезпечує високу продуктивність рендерингу та типобезпеку. Логіка читання та запису JSON-файлів у хмарне сховище S3 (MinIO), реалізована у `worker-mcp-drakon.js`, є стабільною і відповідає галузевим стандартам зберігання неструктурованих даних.¹

3.2. Що придатне для адаптації (Adapt)

Критичної адаптації потребує логіка генерації коду, наразі представлена ендпоінтом `api.generate`. Поточний монолітний "чорний ящик", який очікує від LLM повної побудови діаграми, повинен бути деконструйований у багатоступеневий оркестратор HTSE-конвеєра.¹ Функція застосування дельти (`applyDelta`) у глобальному сховищі `useDiagramStore.ts` повинна бути переписана для імплементації суворої FIFO-черги (First-In-First-Out). Це гарантуватиме, що ітеративні мутації, ініційовані AI-агентом через протокол MCP, не конфліктуватимуть з ручними правками користувача у візуальному редакторі.¹ Крім того, наявна логіка конвертації графа в текст (`pseudocode.ts`) може бути використана як концептуальний базис для зворотного процесу (Code-vs-Diagram sync), але її необхідно розширити від генерації текстових промптів до побудови валідних синтаксичних дерев мов програмування цільового проєкту.¹ Шар аутентифікації також потребує адаптації: зберігання JWT у `localStorage` має бути замінено на використання `HttpOnly Secure cookies` для захисту від XSS-атак.

3.3. Що треба створити з нуля (Build)

Найбільший обсяг робіт пов'язаний зі створенням нових модулів, специфічних для AI-інтеграції. По-перше, необхідно побудувати детермінований AST-екстрактор (Code Analysis Engine). Цей мікросервіс, побудований на базі бібліотеки `ts-morph` або `tree-sitter-graph`, прийматиме сирцевий код і будуватиме граф потоку управління (Control Flow Graph) без участі LLM, гарантуючи 100% відповідність вихідному коду.¹ По-друге, потрібен семантичний транслятор — спеціалізована промпт-система, яка передаватиме цей CFG до LLM виключно для розстановки вказівників (маркування "happy path" та відхилень) без обчислення жодних координат.¹ По-третє, з нуля має бути створений мікросервіс Graph Validator. Це алгоритмічний модуль, який розв'язуватиме задачу задоволення обмежень (Constraint Satisfaction Problem), автоматично зливаючи множинні

вузли повернення (return) у єдиний термінал End, блокуючи заборонені лівобічні вектори та очищуючи граф від вузлів-сиріт.¹

4. Зіставлення поточної архітектури з вимогами Engineering Spec

Інженерна специфікація екосистеми DRAKON чітко артикулює, що інтеграція великих мовних моделей у процеси візуалізації архітектури стикається з фундаментальними топологічними обмеженнями.¹ Класичні інструменти генерації діаграм базуються на алгоритмах гравітаційного макетування або методах балансування маси, які неминуче призводять до візуального хаосу ("спагеті-коду") при обробці складних структур. Для вирішення цієї проблеми специфікація вимагає впровадження парадигми Hybrid Topological Semantic Extraction (HTSE).¹ Зіставлення поточної архітектури aegisroute-ui-starter з цими вимогами демонструє глибокі архітектурні розбіжності, які необхідно усунути.

Ключова невідповідність полягає у відсутності розмежування обов'язків (Separation of Concerns). Згідно з вимогами HTSE, LLM повинна відповідати виключно за семантичне маркування логіки та встановлення вказівників переходів, тоді як координати та топологія обчислюються детермінованим рушієм, а попередній парсинг коду виконується статичними аналізаторами.¹ У поточній системі метод генерації передає сирий текст або код безпосередньо до моделі і очікує повернення кінцевого JSON-графа. Це порушує принцип HTSE і гарантовано призводитиме до структурних помилок, відомих як "просторові галюцинації", коли алгоритм масштабується за межі простих прикладів.¹

Проте існуюча архітектура має одну фундаментально сильну сторону: проміжне представлення даних (Intermediate Representation, IR). Вимога специфікації щодо використання плоского JSON-словника суміжності (adjacency list), де зв'язки є неявними атрибутами вузлів, ідеально імplementована в системі. Інтерфейс DrakonDiagram, що визначає об'єкт items як Record<string, DrakonItem>, повністю відповідає специфікації канонічного IR.¹ Цей формат дозволяє знаходити сусідні вузли за час $O(1)$, і мінімізує структурні помилки під час генерації коду мовними моделями, оскільки їм не потрібно оперувати масивами відокремлених ребер (edges).¹

Топологічна валідація у поточній системі також не відповідає жорстким інваріантам специфікації. Замість алгоритмічного блокування перетинів ліній, зведення множинних кінцевих вузлів до єдиного терміналу та заборони лівобічних векторів для умовних біфуркацій, поточна система використовує метрики лише для спостереження (observability).¹ Метрики на кшталт sis (індикатор єдиного шампура) лише констатують факт порушення правила, але не діють як блокуючий валідатор чи автофіксер, що є неприпустимим для системи, яка повинна гарантувати генерацію математично коректного коду.¹

5. Специфікація цільових підсистем

Для досягнення стану платформи, здатної безпечно та ефективно виконувати структурний рефакторинг реальних кодових баз, необхідно спроектувати та імплементувати п'ять спеціалізованих підсистем. Кожна з них відповідає за окремий етап обробки згідно з парадигмою HTSE.

5.1. Підсистема Code Analysis

Статичний аналіз є фундаментальним першим етапом конвеєра HTSE, оскільки великі мовні моделі не повинні аналізувати сирий текст для вибудовування структурних зв'язків.

- **Інструментарій:** Необхідно розгорнути детермінований парсер. Для проєктів, написаних на TypeScript або JavaScript, оптимальним вибором є ts-morph, який забезпечує глибокий аналіз залежностей та обгортає TypeScript Compiler API. Для розширення платформи на інші мови (Python, Go, Rust) критично важливою стане інтеграція tree-sitter-graph, що дозволяє будувати довільні структури графів із синтаксичних дерев завдяки власній предметно-орієнтованій мові (DSL).¹
- **Генерація CFG:** Завданням цього модуля є читання абстрактного синтаксичного дерева (AST) та його трансформація у граф потоку управління (Control Flow Graph - CFG), який відображає всі можливі шляхи виконання програми, цикли та умови без семантичного забарвлення.
- **Евристичне спрощення:** Аналізатор повинен застосовувати алгоритми спрощення візуального шуму. Наприклад, як утиліта ruflowchart, він має автоматично згортати (collapsing) однорядкові тіла умов (if) або простих циклів у єдиний вузол, а також вирівнювати послідовні умовні оператори для покращення читабельності згенерованої діаграми.¹

5.2. Підсистема Canonical IR (Intermediate Representation)

Канонічне проміжне представлення слугує єдиною мовою спілкування між ШІ-агентом, детермінованим аналізатором та рушієм геометричного макетування.

- **Топологічні вказівники (Implicit Edges):** Відмова від традиційних зв'язків (edges) на користь атрибутів всередині вузлів. Кожен вузол Item повинен містити атрибут one, що вказує на наступний крок по основному вектору виконання (для умов це гілка true, що спрямовується вниз). Атрибут two використовується виключно для альтернативних шляхів (гілка false або обробка помилок catch) і спрямовується строго вправо.¹
- **Валідація схеми:** Структура має підкріплюватися суворою схемою JSON Schema (наприклад, сумісною з draft 2020-12), що гарантуватиме валідність даних перед їх обробкою фронтендом. Це є критичною вимогою для функціонування локальних MCP-серверів.¹
- **Відсутність просторових даних:** IR категорично не повинен містити жодних згадок про координати X та Y. Фізичне розташування є виключною відповідальністю

алгоритму макетування.¹

5.3. Підсистема Validator та Autofix

Мікросервіс Graph Validator відповідає за вирішення задачі задоволення обмежень (Constraint Satisfaction Problem) та блокування будь-яких галюцинацій LLM перед тим, як вони досягнуть візуального рендерера або призведуть до генерації невалідного коду.¹

- **The Single Terminal Constraint:** Валідатор повинен автоматично знаходити всі термінальні вузли (які не мають вказівника one, такі як return або throw), створювати єдиний вузол type: "end" і переписувати всі порожні вказівники на його ідентифікатор. Це гарантує, що алгоритм завжди матиме одну точку логічного завершення.¹
- **Vector Verification (Перевірка векторів):** Жорстка перевірка всіх вузлів типу question або біфуркацій. Якщо модель спробує додати атрибуту переходу, які імплікують рух вліво, валідатор повинен автоматично відхилити таке IR і повернути деталізовану помилку до LLM для ітеративної регенерації.¹
- **Orphan Trimming (Видалення сиріт):** Детермінований алгоритм обходу графа в ширину (BFS) від стартового вузла. Усі вузли, до яких неможливо дістатися по вказівниках one або two, вважаються мертвим кодом або галюцинацією і підлягають автоматичному видаленню перед рендерингом.¹

5.4. Підсистема Diagram Hierarchy (Silhouette)

Масштабування алгоритмів вимагає впровадження ієрархічних макроструктур, щоб запобігти перетворенню складного коду на нечитабельні полотна.

- **Алгоритмічна декомпозиція:** Аналізатор коду повинен визначати цикломатичну складність графа. Якщо складність перевищує встановлений поріг, система відмовляється від побудови простого "Примітиву" і переходить до макроструктури "Силует" (Silhouette).¹
- **Ізоляція гілок (Branches):** Кожен семантично незалежний логічний блок виділяється у паралельну гілку, яка розташовується як окрема колонка дій з власним заголовком та ізольованим "шампуром".¹ Гілки мають розташовуватися зліва направо у хронологічному порядку їх виконання.¹
- **Заборона Anti-Twin Error:** Валідатор повинен суворо забороняти прямі горизонтальні зв'язки між паралельними гілками. Передача управління можлива лише через створення вузла навігації Address, який вказує виключно на ідентифікатор ікони-заголовка (Header) наступної цільової гілки.¹

5.5. Підсистема Code-vs-Diagram Sync

Для забезпечення інтелектуального рефакторингу в реальному часі необхідна система надійної двосторонньої синхронізації між візуальним полотном та вихідним кодом проекту.

- **Iterative Mutations (Ітеративні мутації):** ШІ-агент, що підключається через MCP, не

повинен щоразу переписувати весь JSON діаграми. Система повинна підтримувати передачу гранульованих дельт (наприклад, додавання одного вузла або зміну одного вказівника), які оброблятимуться функцією `applyDelta` у сховищі стану.¹

- **FIFO-серіалізація:** Для уникнення конфліктів стану (race conditions) при паралельному редагуванні (коли інженер переміщує іконку в браузері, а LLM одночасно рефакторить алгоритм), усі події мутації повинні серіалізуватися у строгу чергу First-In-First-Out.¹ Це гарантує цілісність даних та консистентність топології.
- **Зворотне відображення (Diagram-to-Code):** При зміні діаграми розробником (наприклад, перемикання вектора `one` на інший блок), система повинна викликати зворотний генератор, який побудує модифіковане AST та, використовуючи інструмент `MCP drakon.apply_refactoring`, оновить вихідний код у файловій системі.¹ Це замикає цикл двосторонньої синхронізації, перетворюючи малювання на реальне кодування.

6. Цільова архітектура сервісу на 3 фази

Для мінімізації технічних ризиків та забезпечення безперервної поставки цінності, еволюцію проєкту `aegisroute-ui-starter` слід розподілити на три логічні фази.

Фаза 1: MVP (Code Visualization & Prompt Engineering)

На цій фазі система повинна довести здатність стабільно візуалізувати базові фрагменти коду без руйнування топології та "просторових галюцинацій".

- **Архітектурні кроки:**
 1. Розгортання детермінованого AST-парсера (наприклад, `tree-sitter-graph`) на рівні Cloudflare Worker для генерації первинних графів потоку управління (CFG).
 2. Інтеграція LLM виключно для виконання семантичного маркування — переведення CFG у формат канонічного IR з розстановкою вказівників `one` та `two`.
 3. Імплементация мікросервісу Graph Validator для базового автофіксу: злиття множинних вузлів `End` та обрізання вузлів-сиріт.
- **Бізнес-результат:** Користувач може вставити функцію середньої складності (до 100 рядків) і миттєво отримати гарантовано валідну DRAKON-схему (Примітив), яка візуалізує алгоритм без перетинів ліній.

Фаза 2: Production Beta (Bi-directional Sync & MCP Autonomy)

На другій фазі система отримує інтерактивність, здатність обробляти великі масиви коду та можливість двосторонньої взаємодії з агентами.

- **Архітектурні кроки:**
 1. Реалізація алгоритмічної підтримки макроструктури Силует (Silhouette) для автоматичної декомпозиції коду, що перевищує поріг цикломатичної складності.
 2. Розширення MCP Worker новими інструментами, такими як `drakon.update_node` та `drakon.link_nodes`, що дозволить LLM виконувати гранульовані ітеративні

- мутації.
3. Рефакторинг `useDiagramStore.ts` для імплементації черги FIFO, що забезпечить безпечну паралельну роботу розробника та автономного агента.
 4. Активація зворотного процесу: розробка транслятора, який перетворює модифікований JSON-граф назад в AST та записує зміни у вихідний файл.
- **Бізнес-результат:** LLM-агент здатний у фоновому режимі аналізувати логіку, пропонувати оптимізації алгоритму та змінювати діаграму, тоді як інженер бачить ці структурні зміни у візуальному редакторі в реальному часі та може коригувати їх вручну.

Фаза 3: Full Intelligent Refactoring Platform

Третя фаза перетворює інструмент на Enterprise-рішення для масштабного аудиту та модернізації кодових баз.

- **Архітектурні кроки:**
 1. Масштабування AST-аналізатора для крос-файлового аналізу цілих проєктів, відстеження перехресних викликів функцій та управління залежностями.
 2. Впровадження ієрархічної навігації діаграм: реалізація функціоналу, де взаємодія з макроіконою Insertion автоматично завантажує та рендерить внутрішню реалізацію підпрограми як окрему DRAGON-схему.¹
 3. Синхронізація з архітектурними стандартами вищого рівня (наприклад, інтеграція з моделлю C4), де DRAGON-схеми слугуватимуть живим описом найнижчого рівня абстракції (Level 4: Code).
 4. Підключення багатоагентних (Multi-Agent) систем: оркестрація декількох спеціалізованих LLM, які одночасно аналізують топологію на предмет вразливостей безпеки, антипатернів продуктивності та пропонують візуальні рефакторинги.
- **Бізнес-результат:** Повністю автономна платформа, яка здатна реверс-інжинірити legacy-код (наприклад, "спагеті-код"), розбивати його на семантично чисті Силуети, проводити рефакторинг та генерувати оновлену, оптимізовану кодову базу з документацією, що завжди залишається актуальною.

7. Стратегічні артефакти та рекомендації

Нижче наведено практичні артефакти для планування розробки, включаючи класифікацію компонентів, специфікацію нових інструментів MCP та беклог задач.

7.1. Таблиця рішень: Reuse / Adapt / Build

Компонент / Модуль	Статус	Обґрунтування / Необхідна архітектурна
--------------------	--------	---

		дія
drakonwidget.js	Reuse	Математично досконалий рушій, що бездоганно вирішує завдання Манхеттенської маршрутизації та макетування. Використовувати без змін як ядро рендерингу, оскільки він ідеально обробляє плоский JSON. ¹
TanStack & UI (React)	Reuse	Фреймворк, маршрутизація, компоненти Radix та shadcn/ui повністю готові для створення масштабованого та інтерактивного Production-інтерфейсу. ¹
JSON IR Schema	Reuse	Поточний плоский словник ідентифікаторів (items: Record<string, DrakonItem>) є ідеальним канонічним форматом для інтеграції з LLM та забезпечення пошуку $O(1)$. ¹
worker-mcp-drakon.js	Adapt	Критично розширити логіку. Додати обробку ітеративних мутацій (дельта-оновлень) замість існуючого примітивного підходу до повного перезапису об'єкта JSON. ¹
useDiagramStore.ts	Adapt	Впровадити строгу FIFO-серіалізацію подій у

		функцію applyDelta для уникнення race conditions при паралельному редагуванні діаграми людиною та агентом. ¹
pseudocode.ts	Adapt	Значно розширити функціонал від простої генерації текстових промптів до ролі повноцінного транслятора JSON-графа у реальне Абстрактне Синтаксичне Дерево (AST) цільової мови. ¹
HTSE Pipeline	Build	Створити архітектуру суворого розділення: AST-парсинг → LLM для семантики → Graph Validator → drakonwidget для розміщення. ¹
AST Parser Engine	Build	Інтегрувати інструменти рівня ts-morph або tree-sitter у вигляді окремого мікросервісу для детермінованої побудови графів потоку управління з сирцевого коду. ¹
Graph Validator	Build	Створити детермінований алгоритм перевірки інваріантів (злиття терміналів, блокування лівобічних векторів, обрізання вузлів-сиріт). ¹

7.2. Перелік MCP Tools для додавання у воркер

Поточний worker-mcp-drakon.js експонує лише інструменти CRUD (створення, читання,

видалення), що перетворює його на базове файлове сховище.¹ Для розкриття потенціалу інтелектуального рефакторингу необхідно додати специфічні семантичні інструменти:

1. `drakon.analyze_codebase`: Приймає шлях до папки або файлу, виконує AST-парсинг і повертає базовий Control Flow Graph (CFG) для визначених функцій, готуючи фундамент для семантичного маркування ШІ.
2. `drakon.validate_ir`: Приймає згенерований LLM плоский JSON, проганяє його через мікросервіс Graph Validator, повертаючи список автоматичних виправлень (autofixes) або критичних порушень топології, які ШІ має виправити.
3. `drakon.mutate_diagram`: Дозволяє LLM-агенту додавати, видаляти або змінювати (insert, update, delete) окремі вузли в існуючій схемі замість вимоги генерувати весь граф з нуля. Забезпечує підтримку FIFO-черги.
4. `drakon.apply_refactoring`: Приймає модифікований JSON-граф від користувача або ШІ, конвертує його назад в AST та безпечно перезаписує вихідний код у файловій системі проекту розробника, замикаючи цикл синхронізації.

7.3. Беклог задач за пріоритетами

Пріоритет 1: Критичні для стабільності архітектури

- **** Спроектувати та імплементувати мікросервіс Graph Validator для автоматичного злиття множинних вузлів End, блокування лівобічних векторів та видалення вузлів-сиріт (Orphan Trimming).¹
- **** Розгорнути базовий AST-екстрактор (наприклад, `ts-morph` для TypeScript) для детермінованої побудови CFG до того, як код буде переданий у LLM.¹
- **[Frontend]** Рефакторинг стану `useDiagramStore.ts` для впровадження черги обробки подій FIFO, що захистить стан графа від пошкодження під час ітеративних мутацій.¹
- **** Негайно перенести зберігання JWT з `localStorage` у `HttpOnly Secure` куки на рівні `auth.ts` для усунення ризику XSS-атак при роботі з чужим кодом.¹

Пріоритет 2: Функціональне збагачення та інтеграція ШІ

- **[Prompt Engineering]** Розробити жорсткий системний промпт для LLM, що категорично забороняє генерацію координат (X, Y) та зосереджує модель виключно на маркуванні "happy path" через встановлення вказівників `one` та `two`.¹
- **[MCP]** Інтегрувати інструмент `mutate_diagram` в Cloudflare Worker для забезпечення гранульованого ітеративного оновлення діаграм.¹
- **[Frontend]** Налаштувати аналізатор алгоритмічної складності: якщо згенерований CFG перевищує певний поріг кількості вузлів, система повинна примусово декомпонувати алгоритм і будувати макроструктуру "Силует" (Silhouette) замість "Примітиву".¹

Пріоритет 3: Підготовка до Enterprise та масштабування

- **[Architecture]** Реалізувати механізм ієрархічної візуальної навігації, де подвійний клік

на макроікону Insertion ініціює завантаження та рендеринг вкладеної підпрограми як окремої DRAGON-схеми.¹

- **** Створити повноцінний генератор зворотного коду (Diagram-to-AST-to-Code), який дозволить системі виконувати справжній двосторонній структурний рефакторинг.¹

7.4. Ризики та технічні борги

- **Критичний ризик безпеки (XSS Vulnerability):** Поточна імплементація aegisroute-ui-starter зберігає Access Token безпосередньо у localStorage.¹ Оскільки система інтелектуального рефакторингу призначена для парсингу, візуалізації та виконання потенційно небезпечного або невідомого коду та AST-дерев, ризик Cross-Site Scripting є надзвичайно високим. Експлуатація цієї вразливості дозволить зловмиснику миттєво викрасти токен. Перехід на сесійні куки є безальтернативною вимогою.
- **Технічний борг рендерингу (Main Thread Blocking):** Бібліотека drakonwidget.js є потужною, але всі розрахунки складної Манхеттенської маршрутизації виконуються в основному потоці браузера. Для дуже великих Силуетів з сотнями вузлів це може викликати "фризи" та блокування інтерфейсу користувача. Для вирішення цієї проблеми слід розглянути можливість архітектурного винесення розрахунків топології (наприклад, з використанням алгоритмів ELK.js) у Web Worker.¹
- **Ризик "Просторових Галюцинацій" (Hallucination Risk):** Якщо поточний ендпоінт api.generate буде залишено у форматі "чорного ящика" без суворого впровадження HTSE конвеєра, при спробах масштабування на великі кодові бази LLM неминуче почне плутати ідентифікатори вказівників one і two. Це призведе до генерації структурно невалідних "спагеті-графів", що повністю перекреслить цінність переходу на візуальну мову DRAGON.¹

7.5. Стратегічні рекомендації для інтеграції LLM (Claude/Codex/GPT)

Для забезпечення максимальної надійності роботи агентів штучного інтелекту в межах побудованої архітектури необхідно застосувати специфічні техніки взаємодії з LLM.

1. **Контекстне кешування (Context Caching):** Суворі правила топології DRAGON (наприклад, The Skewer Rule, Rightward Degradation) вимагають передачі об'ємних системних промптів. Для моделей на кшталт Claude 3.5 Sonnet слід обов'язково використовувати механізми кешування контексту (Prompt Caching). Це дозволить зберегти специфікацію IR-схеми та правила валідації у пам'яті моделі, значно здешевлюючи вартість ітеративних мутацій та підвищуючи швидкість відповідей під час сесій рефакторингу.
2. **Структурований вивід (Structured Outputs / JSON Mode):** Обов'язкове використання режиму гарантованого JSON-виводу. Модель повинна отримувати

жорстку специфікацію JSON Schema для об'єкта DrakonItem. В атрибутах one та two необхідно реалізувати обмеження через enum або строгі регулярні вирази, які дозволятимуть моделі вводити лише ті ідентифікатори, що вже існують або щойно створені в пулі вузлів, запобігаючи генерації неіснуючих зв'язків.¹

3. **Role Prompting (Семантичний Маршрутизатор):** Штучний інтелект повинен отримати сувору інструкцію щодо своєї ролі. Йому слід прямим текстом заборонити розмірковувати про геометричне розташування або естетику діаграми. Увага моделі має фокусуватися виключно на аналізі бізнес-логіки: визначенні "найбільш імовірного сценарію успіху" (який алгоритмічно стає вказівником one та формує вертикальний шампур) та "шляхів обробки помилок і винятків" (які стають вказівником two і зміщуються праворуч).¹ Модель виступає як інтелектуальний сортувальник намірів розробника, залишаючи всю математику детермінованим рушієм.

Джерела

1. aegisroute-ui-starter_full.pdf
2. MCP + Filesystem is magic : r/ClaudeAI - Reddit, доступ отримано травня 9, 2026, https://www.reddit.com/r/ClaudeAI/comments/1h4yvep/mcp_filesystem_is_magic/
3. Introducing the Model Context Protocol - Anthropic, доступ отримано травня 9, 2026, <https://www.anthropic.com/news/model-context-protocol>
4. Model Context Protocol (MCP) Clearly Explained | by Manoj Desai - Medium, доступ отримано травня 9, 2026, https://medium.com/@the_manoj_desai/model-context-protocol-mcp-clearly-explained-7b94e692001c